

SESSION 6

EXPERIMENT 11 — UNIX DOMAIN SOCKETS

Aim

To implement inter-process communication using **UNIX Domain Sockets** and demonstrate communication between a client and a server process.

Theory

A **UNIX Domain Socket** is an IPC mechanism used for communication between processes running on the same computer.

Unlike network sockets, UNIX Domain Sockets do not use an IP address or network connection. Instead, communication takes place through a **socket file** in the filesystem.

Important concepts:

- **Server:** Creates the socket, binds it to a path, listens for connections and communicates with the client.
- **Client:** Connects to the server socket and sends/receives messages.
- `socket()` — creates a socket.
- `bind()` — assigns a name/path to the socket.
- `listen()` — waits for incoming connections.
- `accept()` — accepts a client connection.
- `connect()` — allows the client to connect to the server.
- `write()` / `read()` — exchange data.
- `close()` — closes the socket.

UNIX Domain Sockets are commonly used by **daemons, background services, Docker, system services and local applications**.

Algorithm

Server

1. Create a UNIX domain socket using `socket()`.
2. Remove an existing socket file if present.
3. Bind the socket to a filesystem path.
4. Listen for client connections.
5. Accept a client connection.
6. Receive a message from the client.
7. Send a response.
8. Close the connection and socket.

Client

1. Create a UNIX domain socket.
2. Connect to the server socket.
3. Send a message.
4. Receive the server response.
5. Display the response.
6. Close the socket.

Code — Server

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/os_server.sock"

int main() {

    int server_fd, client_fd;
    struct sockaddr_un address;
    char buffer[100];

    // Create socket
    server_fd = socket(AF_UNIX, SOCK_STREAM, 0);

    if (server_fd == -1) {
        perror("socket");
        return 1;
    }

    // Remove old socket file
    unlink(SOCKET_PATH);

    memset(&address, 0, sizeof(address));
    address.sun_family = AF_UNIX;
    strcpy(address.sun_path, SOCKET_PATH);

    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address,
             sizeof(address)) == -1) {
```



```
if (bind(server_fd, (struct sockaddr *)&address,
        sizeof(address)) == -1) {
    perror("bind");
    close(server_fd);
    return 1;
}
```

```
// Listen
if (listen(server_fd, 5) == -1) {
    perror("listen");
    close(server_fd);
    return 1;
}
```

```
printf("Server waiting for client...\n");
```

```
// Accept client
client_fd = accept(server_fd, NULL, NULL);
```

```
if (client_fd == -1) {
    perror("accept");
    close(server_fd);
    return 1;
}
```

```
// Receive message
int n = read(client_fd, buffer, sizeof(buffer)) - 
```

```
if (n > 0) {
    buffer[n] = '\0';
    printf("Message from client: %s\n", buffer);
}
```

```
// Send response
char response[] = "Hello from UNIX socket server!";
write(client_fd, response, strlen(response));

close(client_fd);
close(server_fd);
unlink(SOCKET_PATH);

printf("Server terminated.\n");

return 0;
}
```

Code — Client

```
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/os_server.sock"

int main() {

    int client_fd;
    struct sockaddr_un address;
    char buffer[100];

    // Create socket
    client_fd = socket(AF_UNIX, SOCK_STREAM, 0);

    if (client_fd == -1) {
        perror("socket");
        return 1;
    }

    memset(&address, 0, sizeof(address));
    address.sun_family = AF_UNIX;
    strcpy(address.sun_path, SOCKET_PATH);

    // Connect to server
    if (connect(client_fd, (struct sockaddr *)&address,
                sizeof(address)) == -1) {
        perror("connect");
        close(client_fd);
        return 1;
    }
}
```



```

    }

    // Send message
    char message[] = "Hello from UNIX socket client!";
    write(client_fd, message, strlen(message));

    // Receive response
    int n = read(client_fd, buffer, sizeof(buffer) - 1);

    if (n > 0) {
        buffer[n] = '\0';
        printf("Message from server: %s\n", buffer);
    }

    close(client_fd);

    return 0;
}

```

Server output

```

chakrika@Chakrika:~/OSSP-PRACTICALS$ gcc server.c -o server
chakrika@Chakrika:~/OSSP-PRACTICALS$ ./server
Server waiting for client...
Message from client: Hello from UNIX socket client!
Server terminated.
chakrika@Chakrika:~/OSSP-PRACTICALS$

```

Client terminal

```

chakrika@Chakrika:~/OSSP-PRACTICALS$ gcc client.c -o client
chakrika@Chakrika:~/OSSP-PRACTICALS$ ./client
Message from server: Hello from UNIX socket server!
chakrika@Chakrika:~/OSSP-PRACTICALS$

```

EXPERIMENT 12 — SHARED MEMORY IPC

Aim

To implement **Inter-Process Communication using Shared Memory** and demonstrate reading and writing data between processes using a shared memory segment.

Theory

Shared Memory is an IPC mechanism in which multiple processes access a common region of memory.

Instead of sending data through pipes or sockets, processes can directly access the same memory area.

The basic steps are:

1. Create a shared memory segment.
2. Attach the segment to the process's address space.
3. Write or read data.
4. Detach the shared memory.
5. Remove the shared memory segment.

Important system calls/functions include:

- `shmget()` — creates or obtains a shared memory segment.
- `shmat()` — attaches shared memory to a process.
- `shmdt()` — detaches shared memory.
- `shmctl()` — performs control operations such as removing the segment.

Advantages

- Very fast IPC.
- Suitable for large amounts of data.
- Avoids repeatedly copying data through pipes or sockets.

Limitation

Because multiple processes can access the same memory simultaneously, **synchronization** may be required to avoid inconsistent data.

Algorithm

Writer Process

1. Create a shared memory segment.
2. Attach the shared memory.
3. Write data into the shared memory.
4. Display a confirmation.
5. Detach the memory.
6. Exit.

Reader Process

1. Access the existing shared memory.
2. Attach the shared memory.
3. Read the stored data.
4. Display the data.
5. Detach the memory.
6. Remove the shared memory segment.

Code — Writer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {

    key_t key;
    int shmid;
    char *shared_memory;

    // Generate key
    key = ftok(".", 'A');

    if (key == -1) {
        perror("ftok");
        return 1;
    }

    // Create shared memory
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    if (shmid == -1) {
        perror("shmget");
        return 1;
    }

    // Attach shared memory
    shared_memory = (char *)shmat(shmid, NULL, 0);

    if (shared_memory == (char *)-1) {
```

```
// Attach shared memory
shared_memory = (char *)shmat(shmid, NULL, 0);

if (shared_memory == (char *)-1) {
    perror("shmat");
    return 1;
}

// Write data
strcpy(shared_memory,
        "Hello from Writer Process using Shared Memory!");

printf("Writer: Data written to shared memory.\n");

// Detach
shmdt(shared_memory);

return 0;
}
```

Code — Reader

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {

    key_t key;
    int shmid;
    char *shared_memory;

    // Generate same key
    key = ftok(".", 'A');

    if (key == -1) {
        perror("ftok");
        return 1;
    }

    // Access shared memory
    shmid = shmget(key, SHM_SIZE, 0666);

    if (shmid == -1) {
        perror("shmget");
        return 1;
    }

    // Attach shared memory
    shared_memory = (char *)shmat(shmid, NULL, 0);

    if (shared_memory == (char *)-1) {
        perror("shmat");
        return 1;
    }
}
```

```

    if (shared_memory == (char *)-1) {
        perror("shmat");
        return 1;
    }

    // Read data
    printf("Reader: Data received = %s\n", shared_memory);

    // Detach
    shmdt(shared_memory);

    // Remove shared memory
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}

```

OUTPUT:

Writer

```

chakrika@Chakrika:~/OSSP-PRACTICALS$ gcc writer.c -o writer
chakrika@Chakrika:~/OSSP-PRACTICALS$ ./writer
Writer: Data written to shared memory.
chakrika@Chakrika:~/OSSP-PRACTICALS$

```

Reader

```

chakrika@Chakrika:~/OSSP-PRACTICALS$ gcc reader.c -o reader
chakrika@Chakrika:~/OSSP-PRACTICALS$ ./reader
Reader: Data received = Hello from Writer Process using Shared Memory!
chakrika@Chakrika:~/OSSP-PRACTICALS$

```

Result

Thus, Inter-Process Communication using Shared Memory was successfully implemented. The writer process successfully stored data in the shared memory segment, and the reader process successfully accessed and displayed the shared data.